Java OOP Laboratory Exercise

Campus Cafe Ordering System

Name: Jim Nelven B. Cuba

Section: D

November 10, 2026

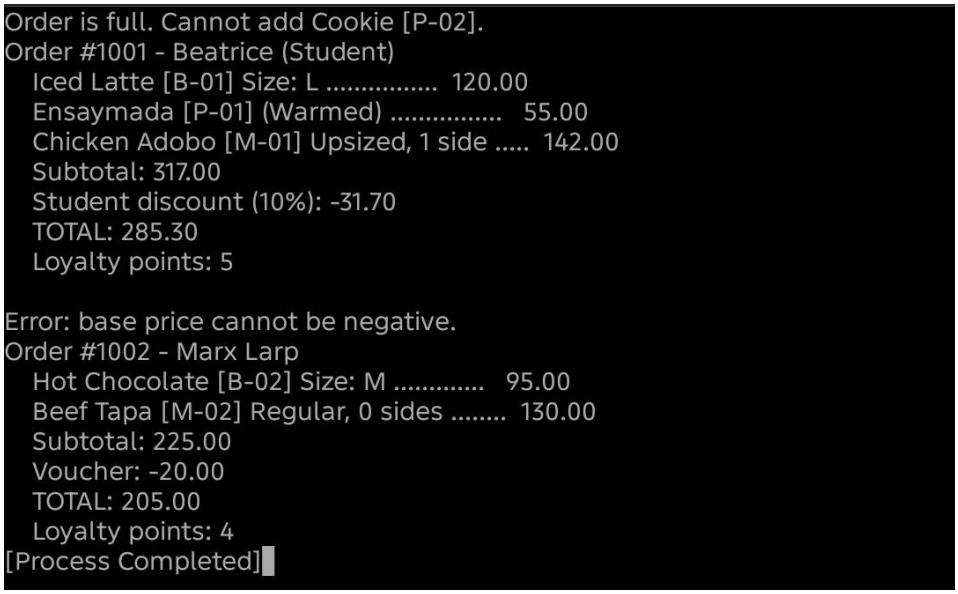
1. Concept Explanation

The program has seven classes: MenuItem (abstract), Beverage, Pastry, Meal, Customer, Order, and CafeApp. The table shows where each OOP concept is used and why.

Concept	Where andwhy it was used
if Statement	Pastry.computePrice() adds the 10-peso warming fee only when the pastry is warmed, and Meal.computePrice() adds the 20-peso upsizesize fee only when the meal is upsized. A single if is enough because there is no other branch.
if-else Statement	MenuItem.setBasePrice()rejectsnegativeprices.Beverage.computePrice() uses an if-else-if chain for sizes S, M, and L. Order.addItem() finds the first empty slot or reports that the order is full, and Order.computeTotal() applies the student discount and the voucher.
Classes and Objects	Eachreal-worldthinginthecafeisaclass.CafeApp.main()createstheobjects:twoCustomers,sixmenu items, and two Orders.
Attributes and Behaviors	Every class has instance variables (such as size in Beverage and loyaltyPoints in Customer) and instance methods (such as computePrice() and addPoints()), so each object keeps its own data and works on it.
Object Interaction	OrderholdsaCustomerandMenuItemobjects.printReceipt()callseachitem'scomputePrice()andthen customer.addPoints() to update the customer's loyalty points.
Java Naming Conventions	Classes use PascalCase (MenuItem), methods and variables use camelCase (basePrice, computeTotal), and constants use UPPER_SNAKE_CASE (WARM_FEE, STUDENT_DISCOUNT_RATE), which keeps the code consistent and easy to read.
Default Constructor	MenuItem(),Beverage(),Pastry(),Meal(),andCustomer()giveobjectssafestartingvalues.HotChocolate and Beef Tapa were created this way and then filled in with setters.
Parameterized Constructor	All MenuItem subclasses, Customer, and Order set their data when the object is created. The subclasses call super(...) to pass the shared name, price, and code to MenuItem.
Constructor Overloading	Customer has three constructors: Customer(), Customer(String name), and Customer(String name, boolean student). They let an object be created with as much information as is available.
public/private/ protected	Methods are public and fields are private. name and basePrice are protected in MenuItem so subclasses can use basePrice directly when computing prices.
Getters,Setters,Data Hiding	There are no public fields. Setters validate input (negative price, negative extra sides, invalid size), so an object cannot hold invalid data. The output line "Error: base price cannot be negative." shows this.
Method Overloading	Customer.addPoints(int) and addPoints(double) share a name but take different inputs; the double version turns pesos into points.Order.computeTotal() and computeTotal (double voucher) work the same way, with the second one subtracting a voucher.
MethodOverriding	Beverage, Pastry, and Meal override computePrice() and getDescription() with @Override. Each type has its own price rule and description, and Order calls them through MenuItem references.
Abstraction	MenuItem is abstract and declares the abstract method computePrice(). It cannot be instantiated and forces every subclass to define its own pricing.

2. Program Output

Output of CafeApp compiled and run with javac and java. The computed values match the expected output: TOTAL 285.30 with 5 loyalty points for Order #1001, and TOTAL 205.00 with 4 loyalty points for Order #1002.



3. Challenges Encountered and How They Were Solved

1. Storing three items without an ArrayList. I used separate fields item1, item2, and item3. addItem() puts the item in the first empty slot with an if-else-if chain and returns false when all three are used, and computeSubtotal() adds only the slots that are not null.

2. Showing the voucher on the receipt. The sample output prints a voucher line, but printReceipt() has no parameter for it. I overloaded printReceipt (double voucher) and made printReceipt() call it with 0.

3. Loyalty points must be whole numbers. I divided the amount by 50 and cast the result to int, so 285.30 pesos gives 5 points and 205.00 pesos gives 4.

4. Validating values inside constructors. The parameterized constructors call the setters, so invalid values are rejected the same way as later changes. MenuItem's constructor first calls this() to start from safe defaults.